

NEXUS-R

Unified Agent-Driven Execution Master Plan

Combining Specification v8.0 with Agent-Native Development Methodology

Version:	8.0 (Unified Synthesis)
Date:	2026-05-18
Sources:	NEXUS-R Master Specification v8.0 + Agent-Driven Execution Plan
Classification:	Technical Specification / Execution Plan
Output Format:	Step-by-Step Codex Prompt Sequences

"The AI agent that gets cheaper the more you use it."

Abstract

This document presents a unified master plan for the NEXUS-R project, synthesizing two complementary documents: the NEXUS-R Master Specification v8.0, which defines the architectural vision, subsystem design, and competitive positioning of a personal agent runtime for progressive cost reduction through verified workflow reuse; and the Agent-Driven Execution Plan, which establishes the methodology for building NEXUS-R entirely through coding agents operating under specification-first, isolation-first, and verification-first principles. The combined plan resolves the critical gap between what to build (the v8 specification) and how to build it using agent-native development (the execution methodology). The central innovation of NEXUS-R is the Execution Trace Distillation (ETD) system: a verified cache of reusable execution plans derived from successful traces, with explicit generalization bounds (>90% success rate threshold) and conservative application criteria (>85% similarity, opt-in). This mechanism makes the system progressively cheaper and faster with each successful execution, creating a defensible moat through accumulated workflow knowledge. The architecture comprises five subsystems (Input Gateway, Cognition Router, Execution Sandbox, State Core, Workflow Engine) and six implementation modules, with a cross-cutting Trust and Control Layer. The implementation roadmap spans four phases over 24+ weeks, with each phase broken down into sequential Codex prompts that can be copy-pasted into the OpenAI Codex agent for autonomous execution. This document provides 33 detailed prompts organized by phase, each with precise specifications, context boundaries, and acceptance criteria, enabling a human operator to orchestrate the entire development process through iterative Codex interactions.

Keywords: Agent-Native Development, Execution Trace Distillation, Capability-Aware Routing, Event-Sourced Architecture, Model Context Protocol, LiteLLM, Local-First Privacy, Workflow Reuse

1. Opening Hook

The fundamental economic problem of AI agent systems is not capability—it is cost. Frontier models like Claude Sonnet 4.1 and GPT-5.5 possess remarkable reasoning abilities, but at \$0.003 per 1K tokens, a developer executing 100 tasks per day accumulates significant expenditure, with the majority of tasks being repetitive variations of previously solved problems. Current agent architectures treat every execution as a novel problem, routing each request to the most expensive model regardless of whether an identical or similar task was successfully completed minutes ago. This is the equivalent of a senior engineer re-deriving a sorting algorithm from first principles every time they encounter an array. The inefficiency

is not theoretical—it is measurable, compounding, and ultimately adoption-limiting.

NEXUS-R proposes a fundamentally different paradigm: a personal agent runtime where every successful execution makes the next one faster and cheaper. Not through subscription discounts or bulk pricing, but through a verified cache of reusable execution plans called Execution Trace Distillation (ETD). When a user deploys a Next.js application to Vercel for the fourteenth time, NEXUS-R does not invoke a frontier model at \$0.003 per token—it retrieves a previously validated, parameterized workflow that executes locally at \$0 cost with sub-5-second latency. The system learns from its own execution history, distills successful traces into minimal causal chains, verifies their generalizability across task variants with quantified bounds (>90% success rate), and applies them conservatively only when environment and intent similarity exceed rigorous thresholds. This is not caching in the traditional sense—it is verified knowledge accumulation with explicit admission criteria and invalidation triggers.

This thesis—that AI task execution can become progressively cheaper through verified workflow reuse—is the core innovation of NEXUS-R. The present document synthesizes the architectural specification (v8.0) with the agent-native development methodology, providing a single, comprehensive master plan organized as a sequence of copy-paste Codex prompts. Each prompt is designed to be executed autonomously by the OpenAI Codex coding agent, with the human operator serving as the Meta-Agent who routes tasks, feeds context, and applies patches. The plan adheres to three inviolable principles: specification is the product (every interface and type must be defined before implementation), agents work in isolation (each agent owns one module with zero ambiguity about boundaries), and verification precedes integration (no module enters the codebase without passing a test suite that a second agent validates). The result is a development process that is itself event-sourced, auditable, and progressively more efficient—mirroring the runtime architecture it produces.

2. Background and Context

2.1 The Agent-Native Development Paradigm

The emergence of autonomous coding agents—OpenAI Codex, Claude Code, Cursor Agent, and similar systems—has fundamentally altered the relationship between specification and implementation in software engineering. When coding agents write 100% of the code, ambiguity becomes fatal. Unlike human developers who can interpret vague requirements, ask clarifying questions, or apply common sense to fill gaps, agents loop on unclear specifications, hallucinate interfaces that do not exist, and ship untested code that appears correct but fails under edge cases. The Agent-Driven Execution Plan identifies this as the central challenge of agent-native development and inverts the traditional approach: specification becomes the primary artifact, implementation becomes a derivative output, and verification becomes a non-negotiable gate.

The paradigm shift has four pillars. First, specification is the product: every interface, type, error code, and test scenario must be defined exhaustively before any implementation code is written. The specification files (.spec.md) become the immutable contracts that constrain agent behavior. Second, agents work in isolation: each agent owns exactly one module with zero ambiguity about boundaries. No agent sees the full codebase; a Meta-Agent (human operator or orchestration script) routes tasks, feeds context, and applies patches. Third, verification precedes integration: no module enters the codebase without passing an agent-written test suite that a second, independent agent validates. Fourth, event-sourced everything: the architecture from the v8 specification (Event Store as single source of truth) is not merely a runtime choice—it is the development methodology. Every agent decision, every test result, and every integration outcome is logged as an immutable event, creating a complete audit trail of the build process itself.

This approach is grounded in recent research on autonomous software engineering. The OpenHands V1 SDK paper [1] establishes event-sourced state management as a production-grade pattern for agent systems, demonstrating that immutable event logs enable replay, debugging, and trajectory extraction. The MoMA framework [2] shows that routing accuracy depends critically on task classification granularity, validating the need for precise specifications that agents can parse without ambiguity. FlowMind [3] demonstrates that trace distillation is feasible but acknowledges that "some fine-grained behaviors may be lost," which NEXUS-R addresses through explicit generalization verification. Shepherd [4] introduces per-step snapshots and trajectory compression with >5x faster forking and >95% cache reuse, techniques that directly inform the ETD pipeline design. Together, these research contributions establish both the feasibility and the challenges of agent-native development, providing the empirical foundation upon which NEXUS-R builds.

2.2 Evolution of the NEXUS-R Specification (v6 through v8)

The NEXUS-R specification has undergone three major revisions, each incorporating hard-won lessons from both internal assessment and external research. Version 6 introduced the original 6-module architecture (Intent Parser, ISE, Execution Engine, CPC, Memory Store, Trust Layer) and identified CPC (Cache of Proven Capabilities) as the strongest novelty claim. However, v6 suffered from over-optimistic timelines, a self-rating inflation, and insufficient attention to the generalizability of cached workflows. The v6/v7 Assessment document delivered a brutally honest 5.0/10 evaluation, cataloging 13 specific technical shortcomings and grounding claims in a research compilation spanning MoMA, OpenHands V1, Claude Code, FlowMind, and Shepherd.

Version 7 responded with a cleaner subsystem redesign (5 subsystems: Input Gateway, CAR, Execution Sandbox, State Core, Workflow Engine), adopted event-sourced single source of truth to replace overlapping memory layers, introduced deny-first security with ML risk classification, and renamed CPC to ETD (Execution Trace Distillation) with explicit generalization verification to address the "vaporware" criticism. However, v7 overcorrected with a self-rating of 9.2/10 that was premature, and the business model remained

underdeveloped. Version 8.0 synthesizes the strongest elements from both predecessors: it retains v7's cleaner subsystem naming and security innovations while preserving v6's implementation granularity and user-facing innovations (cost dashboard, MCP everywhere, LiteLLM integration). The result is an honest 7.5/10 design with a 9/10 core idea, as the v8 specification itself acknowledges.

2.3 Key Definitions and Scope

To ensure clarity throughout this document, the following terms are defined precisely. ETD (Execution Trace Distillation) refers to the process of extracting minimal causal chains from successful execution traces, parameterizing them, verifying their generalizability (>90% success rate across task variants), and admitting them into a conservative reuse cache. CAR (Capability-Aware Router) refers to the multi-objective routing system that considers model capability, latency, cost, and privacy constraints to select the optimal execution path. MCP (Model Context Protocol) is Anthropic's open standard for tool integration [5], which NEXUS-R adopts as the exclusive integration mechanism—no custom connectors are permitted. LiteLLM is the open-source Python SDK and proxy server that provides a unified, OpenAI-compatible interface for 100+ LLM providers [6]. Event Sourcing is the architectural pattern where every state change is captured as an immutable event in an append-only log, enabling replay, debugging, and audit. CPC Short-Circuit refers to the optimization of checking the ETD cache before routing to any model, which is the primary cost-saving mechanism.

The scope of NEXUS-R is deliberately constrained. It is a personal agent runtime—not a platform, not a framework, and not "sovereign cognition." The target users are technical professionals (developers, data scientists, analysts) who execute 50–200 tasks per day across terminal, filesystem, browser, and APIs. The system operates with full cost control, local-first privacy, and learned efficiency. Explicitly out of scope are: Model Council (debating among models is too expensive), World Model (complete state representation is open research), Formal Verification Layer (does not scale; deny-first + ML risk is preferred), Custom Provider Abstraction (LiteLLM already handles 100+ providers), Separate Search/Retrieval Layer (search is a tool via MCP, not a layer), and 12-Layer Architecture (complexity is the enemy of shipping). These anti-requirements are as important as the requirements themselves—they define the boundary between a focused, shippable product and an over-architected system that never compiles.

3. Core Thesis and Architectural Synthesis

3.1 What NEXUS-R Is (and Is Not)

NEXUS-R is a personal agent runtime that makes AI task execution progressively cheaper, more private, and more reliable through intelligent routing and verified workflow reuse. Every successful execution makes the next one faster and cheaper—not because of a subscription

discount, but because the system stops calling expensive APIs for tasks it has already solved. This is the moat: accumulated, verified workflow knowledge that compounds with use, creating switching costs that are structural rather than contractual. When a user has 500 ETDs encoding their most common workflows, migrating to a competitor means losing that accumulated intelligence, which no amount of model quality can immediately replicate.

Equally important is what NEXUS-R is not. It is not "sovereign cognition"—users buy outcomes, not concepts. It is not a platform or framework—it is a runtime that technical professionals use directly. It is not a 12-layer architecture—complexity is the enemy of shipping. It does not include a Model Council, a World Model, a Formal Verification Layer, a Custom Provider Abstraction, or a Separate Search/Retrieval Layer. These anti-requirements were established through hard experience: each represents a feature that previous versions attempted to include but that either failed to ship, added complexity without proportional value, or existed as research questions rather than engineering solutions. The v8 specification's willingness to explicitly enumerate what it is NOT building is a mark of intellectual honesty and engineering discipline.

3.2 The Unified Architecture: 5 Subsystems, 6 Modules

The NEXUS-R architecture adopts v7's subsystem naming (clearer domain separation) while retaining v6's module granularity (more actionable for implementation). The result is a layered architecture comprising five subsystems and six implementation modules, with a cross-cutting Trust and Control Layer that spans all subsystems. The architecture is intentionally flat—no hierarchical coordination, no orchestration layers, no message buses. Each subsystem communicates through the Event Store, which serves as the single source of truth. This design eliminates the synchronization complexity that plagues multi-agent architectures and enables each subsystem to be developed, tested, and deployed independently.

Subsystem	Module Name	Key Responsibilities
1. Input Gateway	Intent Parser	Intent classification, confidence estimation, parameter extraction, complexity scoring (T1-T4)
2. Cognition Router	ISE + CAR	Capability profiling, Pareto routing (cost x latency x privacy), CPC short-circuit, adaptive fallback, quality tracking
3. Execution Sandbox	Execution Engine	Sandboxed agent loop via MCP, inline verification, ML risk classification, permission tiers 1-4, event-sourced traces
4. State Core	Memory Store	Event log (append-only), 3-tier memory (volatile/durable/identity), causal chaining, vector embeddings, projected views
5. Workflow Engine	CPC/ETD	Trace recording, distillation, generalization verification (>90%), parameterization, conservative application (>85%), invalidation
Cross-Cutting	Trust Layer	Permission enforcement, sandbox isolation, audit log, cost dashboard, rollback, secret handling

Table 1: NEXUS-R Unified Architecture — 5 Subsystems, 6 Modules, Cross-Cutting Trust Layer

3.3 Key Design Decisions

The architecture is governed by eight key design decisions derived from both predecessor documents. These decisions are not arbitrary preferences—each is grounded in empirical evidence from prior art and addresses a specific failure mode identified in v6/v7 assessments. Together, they form the architectural contract that constrains all implementation decisions throughout the development process.

Decision	Rationale	Source
MCP everywhere	All tool integrations via MCP. No custom connectors. Reduces integration surface area and ensures interoperability.	implementation_plan
LiteLLM for model abstraction	Don't build provider abstraction. LiteLLM handles 100+ providers with unified interface.	implementation_plan
Event-sourced execution	Every step is immutable event. Enables replay, debugging, CPC extraction.	both
Inline verification	Each step has verify phase. No separate layer. Catches errors at point of occurrence.	implementation_plan
Privacy as hard constraint	Not soft preference. Local models always preferred when privacy flag set.	v7
Deny-first security	ML risk classifier embedded in execution, not bolted on. Default is deny; allow is earned.	v7
CPC short-circuit before routing	Check cache first. Primary cost-saving mechanism.	implementation_plan
Explicit generalization bounds	No workflow applied without >90% proven success rate.	v7

Table 2: Eight Key Design Decisions with Rationale and Source

4. CPC to ETD: The Core Innovation

4.1 What Execution Trace Distillation Is

Execution Trace Distillation (ETD) is a verified cache of reusable execution plans derived from successful traces. It is not "cognitive compression" (which is vague and unmeasurable) and it is not "memory" (which is the wrong category—memory stores information; ETD stores proven execution strategies with quantified reliability). An ETD entry is a structured record containing: the intent signature (what task this workflow solves), the intent embedding (for semantic similarity matching), input and output schemas (typed, validated), a tool sequence (ordered list of tool invocations with inline verification at each step), parameter slots (typed variables that can be substituted across invocations), invariant checks (preconditions that must hold for the workflow to be applicable), and metadata tracking success/failure

counts, generalization success rate, last validation timestamp, average cost and latency, and invalidation triggers.

The distinction between ETD and simpler caching mechanisms is critical. A traditional cache stores input-output pairs: given input X, return output Y. ETD stores execution strategies: given an intent similar to X, here is a verified sequence of tool invocations that, when parameterized with task-specific values, will likely produce the desired output. The "likely" is quantified: ETD entries are only admitted when their generalization success rate exceeds 90% across held-out task variants, and they are only applied when intent similarity exceeds 85% and the current environment satisfies all invariant checks. This conservative approach distinguishes NEXUS-R from prior work like FlowMind, which acknowledges that "some fine-grained behaviors may be lost" during distillation, and OpenHands Condenser, which achieves 2x cost reduction through summarization but does not produce reusable execution plans.

4.2 The ETD Data Structure

The following JSON structure illustrates a complete ETD entry. This structure is the contract that the Workflow Engine module must implement, the Cognition Router must query, and the Execution Sandbox must execute. Every field serves a specific purpose in the distillation, retrieval, application, and invalidation pipeline.

```
{
  "id": "etd_a1b2c3",
  "intent_signature": "deploy-nextjs-to-vercel",
  "intent_embedding": [0.12, -0.05, ...],
  "input_schema": { "project_dir": "string", "env_vars": "map" },
  "output_schema": { "deploy_url": "string", "status": "enum" },
  "tool_sequence": [
    { "tool": "terminal", "action": "npm run build", "verify": "exit_code == 0" },
    { "tool": "terminal", "action": "vercel deploy --prod",
      "verify": "contains(stdout, 'Production')" },
    { "tool": "browser", "action": "GET {deploy_url}",
      "verify": "status == 200" }
  ],
  "parameter_slots": ["project_dir", "env_vars", "deploy_url"],
  "invariant_checks": ["node >= 18", "vercel-cli installed"],
  "model_used": "local/qwen-2.5-7b",
  "success_count": 14,
  "failure_count": 1,
  "generalization_success_rate": 0.94,
  "last_validated": "2026-05-17T10:00:00Z",
  "avg_cost": 0.0,
  "avg_latency_ms": 4200,
  "invalidation_triggers": ["vercel-cli major version change"],
  "derived_from_events": ["evt_001", "evt_002", "evt_003"]
}
```

Figure 1: Complete ETD Entry Data Structure

4.3 The Synthesized ETD Pipeline

The ETD pipeline synthesizes insights from both predecessor documents into a seven-stage process that transforms raw execution traces into conservatively applied, verified workflows. Each stage has precise inputs, outputs, and acceptance criteria, making it suitable for implementation by isolated coding agents working from specifications.

Stage 1: Trace Recording

The Execution Sandbox records every step to the State Core as immutable events. Causal chaining via `parent_event_id` enables precise replay of execution sequences. This stage produces the raw material for distillation and is a direct consequence of the event-sourced architecture decision. Every action—tool invocation, model response, verification result, user approval—is logged with full provenance metadata, including the model used, the cost incurred, the timestamp, and the causal parent. The Event Store's append-only guarantee ensures that traces cannot be retroactively modified, preserving the integrity of the distillation pipeline.

Stage 2: Distillation

The distillation process extracts the minimal causal chain from a successful execution trace. Dead-ends (exploratory steps that did not contribute to the final outcome), retries (failed attempts before the successful path), and exploratory steps (tool invocations that gathered information but did not advance the task) are dropped. The remaining steps represent the essential execution path. Additionally, the distiller identifies which parameters were task-specific (e.g., the project directory path) versus task-agnostic (e.g., the command to run `npm build`) and marks decision points that had alternatives (e.g., choosing between `npm` and `yarn`). This stage is informed by v7's ETD design and addresses FlowMind's acknowledged limitation regarding fine-grained behavior loss by preserving decision-point metadata.

Stage 3: Generalization Verification

This is v7's key improvement and the critical safeguard against the "vaporware" criticism leveled at earlier CPC designs. A distilled workflow is tested on held-out variants of the same task class to measure its success rate across variations. Only workflows achieving >90% generalization success are admitted to the ETD cache. This threshold is not arbitrary: it balances the value of workflow reuse (which increases with admission rate) against the risk of misapplied workflows (which increases with admission rate). The 90% threshold means that at most 1 in 10 applications of an ETD will fail, and the conservative application strategy (Stage 6) ensures that failures are caught early and the system falls back to fresh reasoning without user disruption.

Stage 4: Parameterization and Deduplication

Concrete values in the distilled workflow are replaced with typed parameter slots. For example, `/home/user/project` becomes `{project_dir}` of type `string`. If a new ETD matches an existing intent signature with >80% step overlap, the two are merged, incrementing the success count and updating the generalization success rate. This deduplication prevents the cache from

growing unboundedly with minor variations of the same workflow. The 80% overlap threshold is deliberately high: it ensures that only genuinely equivalent workflows are merged, preserving the specificity that makes ETD valuable. Workflows that differ by more than 20% of steps are treated as distinct entries, allowing the cache to represent genuine task diversity without conflating different execution strategies.

Stage 5: Retrieval

ETD retrieval uses a multi-factor ranking algorithm. First, intent matching is performed via cosine similarity on embeddings, identifying candidate workflows whose intent signatures are semantically similar to the current request. Second, context gating checks the invariant checks of each candidate against the current environment (are the required tools installed? do version constraints hold?). Third, candidates are ranked by $\text{success_rate} * \text{recency_weight}$, favoring workflows with proven reliability and recent validation. Fourth, a hard threshold excludes any ETD with similarity $< 85\%$ or success rate $< 90\%$. The result is either a single best-match ETD (if one candidate exceeds all thresholds) or no match (triggering fresh routing through the Cognition Router).

Stage 6: Conservative Application

Even after passing retrieval thresholds, ETD application follows conservative safeguards derived from v7. The feature is opt-in: users must explicitly enable workflow suggestions. Environment matching is required: the same tools must be available in the current context. If any step in the ETD fails its inline verification, the system abandons the ETD immediately and falls back to fresh reasoning through the Cognition Router—there is no attempt to "patch" a failing ETD mid-execution. After execution, the output schema is validated against the ETD's declared `output_schema`, and success/failure counts are updated. This post-execution validation creates a feedback loop that continuously refines the cache's reliability estimates.

Stage 7: Invalidation

ETD entries are not immortal. They are subject to three invalidation triggers. First, if the failure rate exceeds 30% (i.e., $\text{failure_count} / (\text{success_count} + \text{failure_count}) > 0.3$), the entry is marked as degraded and excluded from retrieval until its success rate recovers (which requires new successful executions). Second, if an invalidation trigger fires (e.g., a dependency version change detected by the Execution Sandbox), the entry is immediately invalidated and must be re-derived from new execution traces. Third, entries unused for 180 days are archived (not deleted—the append-only Event Store preserves the derivation history). User manual invalidation is also supported, providing an escape hatch for entries that are technically valid but contextually inappropriate.

4.4 Research Positioning and Prior Art

The research claim for NEXUS-R ETD is: execution trace distillation with verified generalization bounds for autonomous workflow reuse. The novelty is not "compressing

traces"—prior systems achieve compression. The novelty is admitting only provably generalizable plans into a conservative reuse system with quantified reliability guarantees. The following table positions NEXUS-R relative to the most relevant prior art, identifying what each system achieves and what NEXUS-R adds beyond existing capabilities.

Prior Art	What They Do	What NEXUS-R ETD Adds
FlowMind [3]	Distill traces to structured workflows via Execute-Summarize framework	Explicit generalization verification with quantified bounds; conservative application thresholds
Shepherd [4]	Per-step snapshots, trajectory compression, >5x faster forking, >95% cache reuse	Conservative application thresholds + causal chaining for provenance; workflow reuse (not just compression)
OpenHands Condenser [1]	2x cost reduction via summarization of execution history	Reusable execution plans (not just compressed history); parameterized workflow templates
Deep Memory	71% token reduction via bio-inspired compression	Workflow reuse (not just context compression); structural templates with verification

Table 3: Research Positioning — NEXUS-R ETD vs. Prior Art

5. ISE to CAR: Capability-Aware Routing

5.1 The Routing Problem

The core routing problem in multi-model agent systems is that "sufficiency" is undecidable a priori: you only know whether a model is capable of handling a task after trying. MoMA [2] demonstrates that routing accuracy depends critically on task classification granularity, while OpenHands RouterLLM requires a custom `select_llm()` function per use case. The cost of failed weak-model attempts may exceed the savings from successful ones, creating a negative expected value for aggressive routing. The v8 specification synthesizes insights from both predecessor documents to produce a seven-step routing solution called Capability-Aware Router (CAR) with CPC Short-Circuit and Parallel Probes.

5.2 The CAR Solution

Step 1: CPC Check. Before any routing, the ETD cache is consulted. If a high-confidence match exists (similarity >85%, success rate >90%), the task is executed directly from the cached workflow, typically at \$0 cost for local ETDs. This is the primary cost-saving mechanism and the reason ETD is the core innovation: the routing problem is moot for cached tasks.

Step 2: Task Classification. Tasks are scored along three dimensions: reasoning depth (logical steps, ambiguity, planning need, 40% weight), knowledge breadth (domain specificity, recency, 30% weight), and tool complexity (number/type of tools, irreversibility, 30% weight). A small, fast classifier (rule-based initially, fine-tuned small model later) scores each dimension 1-4, producing a composite task tier (T1-T4). This multi-dimensional classification avoids the

binary "simple/complex" categorization that MoMA [2] identifies as insufficient for accurate routing.

Step 3: Capability Profiling. Each available model maintains a profile including capability scores (code, math, writing, reasoning), latency percentiles (p50, p99), cost per 1K tokens, privacy level (local/private_server/cloud), tool format compatibility, and context window size. Profiles are updated based on observed outcomes, creating a feedback loop that improves routing accuracy over time. This is v7's key addition: rather than relying on static model specifications, CAR learns from actual execution results.

Step 4: Multi-Objective Routing. CAR optimizes across five dimensions: capability match (estimated vs. observed), latency requirement, cost constraint, privacy requirement (hard constraint, not preference), and verification confidence. The privacy dimension is a hard constraint: if the user has set a privacy flag, cloud models are excluded regardless of capability or cost advantages. This reflects the design decision that "privacy is not a soft preference" but a hard constraint.

Step 5: Adaptive Fallback Chain. The fallback chain escalates through six tiers: local_7b (\$0, max privacy) to local_14b (\$0, max privacy) to local_70b (\$0, max privacy) to BYOK_budget (\$user_cost, user control) to BYOK_frontier (\$user_cost, user control) to managed_premium (\$platform_cost, standard privacy). Cost caps apply at each tier. The chain never jumps more than one tier at a time, and the managed_premium tier always requires explicit user approval. This gradual escalation minimizes cost while ensuring capability.

Step 6: Parallel Probe. When the classifier is uncertain about the appropriate tier, the task is sent to T1 and T2 simultaneously. If T1 succeeds verification, the T2 result is discarded. The cost of this probe is low because T1 is local and free. This technique addresses the "sufficiency is undecidable a priori" problem by empirically testing the cheaper option while hedging with the more capable one.

Step 7: De-escalation Learning. If a task consistently succeeds at T1 after being classified as T2, the classifier weights are updated to route similar tasks to T1 in the future. Over time, as ETD accumulates and the classifier improves, an increasing proportion of tasks route locally—achieving the project's goal of progressive cost reduction without manual tuning.

5.3 Performance Targets

Metric	Target	Rationale
Routing overhead	<50ms	Research confirms routing is cheap vs. LLM inference (2–15s)
End-to-end latency (simple)	<3s	Flat architecture, no hierarchical coordination
Cost reduction vs. always-frontier	60–80%	MoMA and ICA AI demonstrate this range
Local mode capability	80% of cloud on common tasks	70B models + ETD reuse for standard patterns

Table 4: CAR Performance Targets

6. Security and Trust Layer

6.1 Permission Model: Capability-Based, 4 Tiers

The permission model implements capability-based security with four tiers of increasing privilege. The design follows a deny-first philosophy: actions are denied by default, and permission is granted only when explicitly justified by the task classification and the user's trust configuration. Tier 1 (Auto) permits read-only operations: reading files, read-only terminal commands (ls, cat, grep), web search, and safe MCP tools. Tier 2 (Auto + log) permits write operations within the workspace: modifying workspace files, sandboxed terminal commands, and HTTP GET to approved domains. Tier 3 (User prompt) permits operations with external side effects: arbitrary terminal commands, accessing files outside the workspace, HTTP POST/PUT/DELETE, browser actions, and API key usage. Tier 4 (Explicit confirm + reason) permits irreversible operations: deleting files, accessing secrets, deploying to production, financial transactions, and any action with permanent consequences.

Tier	Actions	Approval
T1	Read files, read-only terminal, web search, safe MCP tools	Auto
T2	Write/modify workspace files, sandboxed terminal, HTTP GET to approved domains	Auto + log
T3	Arbitrary terminal, files outside workspace, HTTP POST/PUT/DELETE, browser actions, API keys	User prompt
T4	Delete files, access secrets, deploy to prod, financial transactions, irreversible actions	Explicit confirm + reason

Table 5: Four-Tier Permission Model

6.2 Sandboxing

Sandboxing is implemented at multiple levels to provide defense-in-depth. Terminal commands execute within Docker containers with only the workspace directory mounted; if Docker is unavailable, OS-level restrictions (chroot, namespaces) serve as fallback. Browser automation uses an isolated Chromium instance with no host filesystem access and no real browser profile, preventing credential leakage and session hijacking. Filesystem access is scoped to the workspace directory; parent directory traversal is explicitly blocked. API calls are restricted to whitelisted domains, with credential injection handled by a SecretRegistry that never exposes secrets to the model layer. This multi-layered approach ensures that even if one sandbox boundary is breached, the attacker (or compromised agent) cannot escalate to the host system or access resources outside the designated scope.

6.3 ML Risk Classification

The ML risk classifier evaluates each proposed action before execution, computing a risk score based on action type, target resource, permission scope, and user history. Low-risk actions are auto-executed; medium-risk actions prompt the user for approval; high-risk actions require explicit override with a documented reason and audit log entry. The classifier runs locally (no cloud dependency for security decisions) with a target latency of less than 10ms per action, ensuring that security checks do not introduce perceptible delay. The risk classification model is initially rule-based (Phase 1-2) and is later refined with observed outcomes (Phase 3+), creating a feedback loop that improves accuracy over time while maintaining the deny-first philosophy.

6.4 Audit and Rollback

Every action is recorded in an immutable audit log with the following fields: timestamp, action_type, tool, input_summary, output_summary, model_used, tier, cost, user_approved, verification_result, and action_hash. File operations create snapshots before write/delete operations (using git stash or file copies), enabling rollback to any previous state. Terminal commands record stdout/stderr and provide undo suggestions when available. Audit log retention is 90 days by default and user-configurable. The audit log is exportable for compliance and forensics, and it is append-only—no deletion or modification is permitted, ensuring a complete chain of custody for every action the system takes.

6.5 Prompt Injection Defense

Prompt injection is addressed through four layers of defense. First, input framing: external content is wrapped in <EXTERNAL_DATA> delimiters, signaling to the model that the enclosed content should not be treated as instructions. Second, output validation: all model outputs are parsed structurally (JSON tool calls), never evaluated as code (no eval()), preventing injected commands from being executed. Third, action allowlisting: only explicitly declared tools are callable; dynamic tool creation is prohibited, eliminating the attack surface of "tool injection" through crafted prompts. Fourth, anomaly detection: if a TIER 4 request occurs during a TIER 1 task (e.g., a request to delete files during a read-only search), the action is flagged and paused for manual review. This multi-layered defense ensures that even if one layer is bypassed, subsequent layers catch the attack.

7. State Core: Three-Tier Memory

The State Core implements a three-tier memory architecture that separates concerns based on lifecycle, access patterns, and persistence requirements. The design adopts v7's key insight that the Event Store should be the single source of truth, with all other memory tiers derived from it as projected views. This eliminates the synchronization problems that plagued earlier

multi-layer memory architectures and ensures that any state can be reconstructed from the event log.

Tier	Name	What It Stores	Lifecycle	Storage
Volatile	Working State	Current task context, active plan, tool results, conversation	Dies when task completes	In-memory
Durable	Event Store	Immutable log of all actions, decisions, outcomes, ETD derivations	Append-only, 90 days default	Local SQLite + optional sync
Identity	User Profile	Preferences, tool configs, API key refs, behavioral policies, classifier weights	Slow-changing, user-editable	Local encrypted file

Table 6: Three-Tier Memory Architecture

Key design features include causal chaining via `parent_event_id`, which links events into provenance chains that enable precise replay and debugging; vector embeddings for every event, enabling semantic retrieval across the entire execution history; projected views that provide read-optimized representations for UI, billing, audit, and debugging without duplicating state; and Deep Memory-style semantic compaction, which achieves approximately 71% token reduction demonstrated in prior work. What was deliberately removed includes the World Model (too ambitious, replaced by scoped task state in Working State), the Personal Cognition Layer (merged into Identity tier + ETD), and the Separate Persistent Workflow Memory (ETD is derived from Event Store, not a separate layer). These removals reflect the v8 specification's commitment to shipping a focused product rather than an over-architected system.

8. Agent Team Topology and Role Definitions

The Agent-Driven Execution Plan establishes a specialized team topology where each agent role has a constrained context window, explicit input/output formats, and ownership over a single, well-defined responsibility domain. This is not a "frontend/backend" split—it is a granular decomposition of the development process into roles that can be executed by coding agents with minimal ambiguity. The orchestration rule is strict: no agent sees the full codebase. A Meta-Agent (the human operator using Codex) routes tasks, feeds context, and applies patches. Each Codex prompt in the execution roadmap is designed for exactly one agent role, with context boundaries that prevent information leakage between modules.

Agent Role	Context Window	Responsibility	Input	Output
Spec Architect	200k tokens	Writes .spec.md files: interfaces, types, error codes, test scenarios	v8 subsystem docs, research papers	Module specifications + acceptance criteria

Foundation Engineer	100k tokens	Scaffolds repo structure, CI/CD, base types, event schemas	Specs for Phase 1	Python package skeleton + SQLite schema + MCP boilerplate
Module Implementer	100k tokens	Writes business logic for ONE subsystem	.spec.md + foundation code	Module code + unit tests
Test Generator	100k tokens	Writes adversarial tests, property-based tests, chaos scenarios	.spec.md + implementation	test_*.py with >=90% coverage target
Integration Validator	150k tokens	Verifies cross-module contracts, writes integration tests	All module specs + stubs	Integration test suite + contract verification
Security Auditor	100k tokens	Reviews for injection vectors, sandbox escapes, secret leaks	Implementation + specs	Security audit report + hardened patches
ETD Distiller	100k tokens	Specialized agent for Workflow Engine/CPC logic	Event schemas + execution traces	ETD extraction + generalization logic

Table 7: Agent Team Topology — Seven Specialized Roles

The seven-agent topology is designed to minimize context window waste and maximize specification adherence. The Spec Architect operates at 200k tokens because specification writing requires referencing extensive architectural documentation and research papers. The Foundation Engineer operates at 100k tokens because the initial scaffolding is self-contained. Module Implementers each operate at 100k tokens with access only to their module's specification and the foundation code—never to other modules' implementations. The Test Generator receives specifications and implementations to produce adversarial tests targeting >=90% coverage. The Integration Validator operates at 150k tokens because cross-module contract verification requires understanding multiple specifications simultaneously. The Security Auditor reviews implementations and specifications for security vulnerabilities. The ETD Distiller is a specialized agent with deep knowledge of the Workflow Engine and event schemas, focused exclusively on the core innovation of the system.

9. Repository Structure and Agent Constraints

The repository structure is designed to be agent-navigable: each directory maps to a clear ownership boundary, and every module follows a consistent internal structure that agents can parse without ambiguity. The top-level directory layout is as follows:

```
nexus-r/
+-- specs/ # Source of truth. Agents read, never write.
| +-- 00_architecture.md
| +-- 01_input_gateway.spec.md
| +-- 02_cognition_router.spec.md
| +-- 03_execution_sandbox.spec.md
| +-- 04_state_core.spec.md
```



```

| +-- 05_workflow_engine.spec.md
| +-- 06_trust_layer.spec.md
| +-- 99_acceptance_criteria.md
+-- foundation/ # Written by Foundation Engineer
| +-- nexus_r/
| | +-- __init__.py
| | +-- events.py # Event dataclasses + causal chaining
| | +-- models.py # LiteLLM abstraction
| | +-- config.py # Pydantic settings
| | +-- exceptions.py # Hierarchy of NEXUS exceptions
| +-- pyproject.toml
| +-- Dockerfile.sandbox
| +-- .github/workflows/ci.yml
+-- modules/ # One directory per subsystem
| +-- input_gateway/
| +-- cognition_router/
| +-- execution_sandbox/
| +-- state_core/
| +-- workflow_engine/
| +-- trust_layer/
+-- tests/
| +-- unit/ # Written by Test Generator per module
| +-- integration/ # Written by Integration Validator
| +-- security/ # Written by Security Auditor
| +-- fixtures/ # Shared test data (immutable)
+-- e2e/ # End-to-end scenarios
+-- scripts/
+-- agent_task_router.py # Meta-Agent orchestration
+-- verify_module.py # Entrypoint for CI gate

```

Figure 2: Repository Directory Structure

A critical agent constraint governs every module directory: each **MUST** contain a `README.md` (10-line summary of module contract), a `src/` directory (implementation), a `tests/` directory (unit tests runnable via `pytest` independently), and an `interface.yaml` file (declared dependencies on other modules and external packages). This constraint ensures that any agent can understand a module's contract by reading `README.md`, verify its correctness by running `tests/`, and understand its integration requirements by reading `interface.yaml`—without needing to read the implementation code. The `specs/` directory is the source of truth that agents read but never write; it is produced by the Spec Architect agent and remains immutable throughout the development process.

10. Implementation Roadmap with Step-by-Step Codex Prompts

This section constitutes the core of the master plan: a sequential set of 33 Codex prompts organized by phase, each designed to be copy-pasted into the OpenAI Codex agent for autonomous execution. The human operator serves as the Meta-Agent, pasting one prompt at a time, waiting for Codex to complete execution, reviewing the output, and then pasting the next prompt. Each

prompt includes: the agent role being invoked, the precise task specification, the context that Codex should reference, the expected output format, and the acceptance criteria that determine whether the prompt's execution is complete. The prompts are designed to be self-contained: Codex should not need to reference any external documentation beyond what is provided in the prompt itself.

How to Use These Prompts: (1) Open the OpenAI Codex agent (CLI or web interface). (2) Copy the entire text from a PROMPT box below. (3) Paste it into Codex and wait for execution to complete. (4) Review the output against the stated acceptance criteria. (5) If the criteria are met, proceed to the next prompt. If not, paste the prompt again with additional clarification. (6) After each prompt, verify that the expected files have been created/modified in the repository before proceeding. (7) Do NOT skip prompts or execute them out of order—each prompt assumes the output of all previous prompts is available.

10.1 Phase 1: Core Runtime (Weeks 1–8)

Goal: Can a user give a natural language task and have it executed locally or via BYOK, with full audit trail? This phase builds the minimum viable product: a CLI tool that accepts natural language tasks, routes them through a basic 2-tier CAR (local vs. BYOK), executes them in a sandboxed environment via MCP, logs every action to an event-sourced SQLite database, and tracks costs per request. No ETD, no browser automation, no web UI, no managed premium tier. The team is 2–3 engineers (or, in the agent-native paradigm, one human operator orchestrating Codex prompts). Validation: 5 beta users run 20+ tasks each; measure success rate and cost.

CODEX PROMPT 1: Initialize Repository + Foundation Structure

You are the Foundation Engineer agent for the NEXUS-R project. Your task is to initialize the complete repository structure for a Python project called "nexus-r". CONTEXT: NEXUS-R is a personal agent runtime that makes AI task execution progressively cheaper through intelligent routing and verified workflow reuse. It uses MCP for tool integration, LiteLLM for model abstraction, and event-sourced architecture with SQLite. INSTRUCTIONS: 1. Create the full directory structure: nexus-r/ ├── specs/ ├── foundation/nexus_r/ ├── modules/{input_gateway,cognition_router,execution_sandbox,state_core,workflow_engine,trust_layer}/ ├── tests/{unit,integration,security,fixtures}/ ├── e2e/ └── scripts/ 2. In foundation/nexus_r/, create: - __init__.py (version 0.1.0) - events.py: Define dataclasses for Event, CausalEvent (with parent_event_id), EventStore (SQLite-backed, append-only), with methods: append(), query(), get_chain() - models.py: LiteLLM abstraction class ModelRouter with method complete(model, messages, **kwargs) that wraps litellm.completion - config.py: Pydantic Settings class NEXUSConfig with fields for db_path, default_model, privacy_mode (bool), cost_cap (float), mcp_server_url - exceptions.py: Hierarchy: NEXUSError → {RoutingError, ExecutionError, VerificationError, SecurityError, ETDError} 3. Create pyproject.toml with dependencies: litellm, pydantic, pydantic-settings, mcp, pytest, pytest-asyncio, aiosqlite 4. Create Dockerfile.sandbox: Python 3.12 slim, install project, expose no ports, mount /workspace 5. Create .github/workflows/ci.yml: run pytest on push/PR to main ACCEPTANCE CRITERIA: - All directories and files exist - `python -c "from nexus\_r.events import Event, EventStore"` succeeds - `python -c "from nexus\_r.models import ModelRouter"` succeeds - `python -c "from nexus\_r.config import NEXUSConfig"` succeeds - `python -c "from nexus\_r.exceptions import NEXUSError, RoutingError"` succeeds - SQLite EventStore can append and query events - Dockerfile builds without errors

CODEX PROMPT 2: Spec Architect — Write All Phase 1 Specifications

You are the Spec Architect agent for the NEXUS-R project. Your task is to write exhaustive specification files (.spec.md) for the Phase 1 modules. These specs are the IMMUTABLE contracts that implementation agents will follow. CONTEXT: NEXUS-R v8.0 architecture has 5 subsystems: Input Gateway, Cognition Router, Execution Sandbox, State Core, Workflow Engine, plus a cross-cutting Trust Layer. Phase 1 builds: Input Gateway (basic), Execution Sandbox (terminal+files only), basic Cognition Router (2-tier), State Core (event log), and Trust Layer (T1-T2 permissions only). INSTRUCTIONS: Create the following files in specs/: 1. 00_architecture.md: System overview, subsystem boundaries, data flow diagrams (ASCII), event schema, error handling philosophy, module dependency graph 2. 01_input_gateway.spec.md: Intent classification (regex+keyword, categories: code/research/automation/configuration), confidence estimation (0-1 float), parameter extraction (JSON schema output), complexity scoring (T1-T4 dimensions: reasoning_depth 40%, knowledge_breadth 30%, tool_complexity 30%). Define all function signatures, input/output types, error codes (IG-001 to IG-050). 3. 02_cognition_router.spec.md: Basic CAR with 2 tiers only (local_7b via Ollama, BYOK via LiteLLM). CPC short-circuit (query EventStore for matching ETD). Fallback: local → BYOK. Define provider_profile schema, routing_decision schema, cost_cap enforcement. Error codes CR-001 to CR-050. 4. 03_execution_sandbox.spec.md: MCP-based execution loop (terminal + filesystem tools only). Inline verification after each step. Permission tiers T1-T2 (deny-first). Event-sourced trace: every action logged to EventStore. Docker sandbox for terminal. Workspace scoping for filesystem. Define execution_step schema, verification_result schema, sandbox_config schema. Error codes ES-001 to ES-080. 5. 04_state_core.spec.md: EventStore (SQLite, append-only, immutable). Event schema with parent_event_id for causal chaining. Volatile Working State (in-memory dict, dies with task). Durable Event Store (SQLite). Identity tier (encrypted JSON file). Define event types, query API, compaction strategy. Error codes SC-001 to SC-050. 6. 05_workflow_engine.spec.md: Phase 1 = trace recording only. No ETD extraction yet. Define trace recording from Execution Sandbox events, minimal event schemas for workflow derivation. Error codes WE-001 to WE-030. 7. 06_trust_layer.spec.md: T1-T2 permissions only. Permission checking before every action. Audit logging (append to EventStore). Cost tracking per request (no dashboard). Secret handling (OS credential manager). Define permission_check schema, audit_entry schema, cost_record schema. Error codes TL-001 to TL-050. 8. 99_acceptance_criteria.md: Phase 1 acceptance criteria: 5 beta users, 20+ tasks each, >80% success rate, average cost < \$0.10 for T1-T2, full audit trail for every action. ACCEPTANCE CRITERIA: - All 8 spec files exist in specs/ directory - Each spec defines ALL function signatures with typed parameters and return types - Each spec defines ALL error codes with descriptions - Each spec includes at least 5 test scenarios - No spec references implementation details (only interfaces and contracts) - Total spec word count > 10,000 words across all files

CODEX PROMPT 3: Foundation Engineer — Scaffold Infrastructure and Base Types

You are the Foundation Engineer agent for the NEXUS-R project. Your task is to implement the shared infrastructure that all modules depend on. CONTEXT: Read specs/00_architecture.md for the system overview. The foundation layer provides event schemas, model abstraction, configuration, and exception hierarchy that all modules will import. INSTRUCTIONS: 1. Enhance foundation/nexus_r/events.py: - Event dataclass: id (UUID), timestamp (datetime), event_type (str), parent_event_id (Optional[UUID]), data (dict), embedding (Optional[list[float]]) - CausalEvent extends Event with verification_result, model_used, cost, tier - EventStore class: SQLite-backed (use aiosqlite for async), methods: append(event), query(filters), get_chain(event_id), get_latest(n), create_projected_view(view_name, query) - Implement causal chain traversal: given event_id, return full chain of parent_event_ids back to root 2. Enhance foundation/nexus_r/models.py: - ModelRouter: wraps litellm.completion, handles provider profiles, cost tracking - ProviderProfile dataclass: model_id, capabilities (dict), latency_p50_ms, cost_per_1k_tokens, privacy_level, tool_format, context_window - Method: select_model(task_tier, privacy_required, cost_cap) returns best model from available profiles - Method: complete(model_id, messages, **kwargs) with automatic retry and cost logging 3. Enhance foundation/nexus_r/config.py: - NEXUSConfig with all settings from spec - Support .env file loading - Validation: cost_cap > 0, db_path writable, default_model in available models 4. Create scripts/verify_module.py: - CLI tool: python verify_module.py - Runs pytest on the specified module's tests/ - Reports coverage percentage - Returns exit code 0 if coverage >= 80%, else 1 5. Create scripts/agent_task_router.py: - Simple CLI for Meta-Agent: lists pending tasks, marks tasks complete, shows module status ACCEPTANCE CRITERIA: - EventStore can append 1000 events in < 1 second - Causal chain traversal returns correct parent chain - ModelRouter.select_model returns local model when privacy_required=True - verify_module.py works for any module in modules/ - All foundation unit tests pass with >= 80% coverage

CODEX PROMPT 4: Module Implementer — Input Gateway

You are the Module Implementer agent for the Input Gateway subsystem of NEXUS-R. CONTEXT: Read specs/01_input_gateway.spec.md for the complete specification. This is Phase 1 (basic implementation). INSTRUCTIONS: Implement modules/input_gateway/ with the following structure: - README.md (10-line module contract) - src/__init__.py, intent_parser.py, classifier.py, parameter_extractor.py - tests/test_intent_parser.py, test_classifier.py, test_parameter_extractor.py - interface.yaml (dependencies: nexus_r.events, nexus_r.config) Implementation details: 1. intent_parser.py: IntentParser class with method parse(user_input: str) → IntentResult - IntentResult: intent_type (code/research/automation/configuration), confidence (0-1), parameters (dict), complexity_tier (T1-T4) - Phase 1: Use regex patterns + keyword matching for classification - Complexity scoring: reasoning_depth (count logical connectors, question marks), knowledge_breadth (domain-specific terms), tool_complexity (count tool-referencing words) 2. classifier.py: TaskClassifier class with method classify(intent_result: IntentResult) → TaskTier - Returns T1-T4 based on weighted scoring: reasoning_depth*0.4 + knowledge_breadth*0.3 + tool_complexity*0.3 3. parameter_extractor.py: ParameterExtractor with method extract(user_input: str, intent_type: str) → dict - Extract file paths, command names, URLs, and key-value pairs from input - Return structured dict with typed values 4. Write comprehensive unit tests targeting >= 90% coverage 5. Create interface.yaml listing dependencies ACCEPTANCE CRITERIA: - parse("deploy my app to vercel") returns intent_type="automation", complexity_tier in [T2, T3] - parse("what is the capital of France") returns intent_type="research", complexity_tier=T1 - parse("delete all log files in /var/log") returns intent_type="code", complexity_tier=T4 - All unit tests pass - Coverage >= 90% - interface.yaml is valid YAML

CODEX PROMPT 5: Module Implementer — Execution Sandbox

You are the Module Implementer agent for the Execution Sandbox subsystem of NEXUS-R. CONTEXT: Read specs/03_execution_sandbox.spec.md. Phase 1: terminal + filesystem tools only, no browser. INSTRUCTIONS: Implement modules/execution_sandbox/ with: - README.md, src/, tests/, interface.yaml Implementation: 1. sandbox.py: ExecutionSandbox class - __init__(config: NEXUSConfig, event_store: EventStore) - execute(task: TaskDefinition) → ExecutionResult - Uses MCP protocol for tool invocations (terminal, filesystem) - Docker sandbox for terminal commands (fallback: subprocess with restricted permissions) - Workspace scoping: all filesystem ops restricted to config.workspace_path 2. mcp_client.py: MCPClient class - Connects to MCP server for tool discovery and invocation - Methods: list_tools(), invoke_tool(tool_name, params) → ToolResult - Implements MCP protocol messages (initialize, tools/list, tools/call) 3. verifier.py: InlineVerifier class - verify_step(step: ExecutionStep, result: ToolResult) → VerificationResult - Checks: exit_code == 0 for terminal, file_exists for filesystem writes, content_matches for reads - Returns pass/fail with detailed message 4. permission_checker.py: PermissionChecker class - check_permission(action: Action, tier: PermissionTier) → PermissionDecision - T1: allow read-only operations - T2: allow workspace writes, log action - T3+: deny (Phase 1 does not support T3-T4) 5. trace_logger.py: TraceLogger class - Logs every action to EventStore as CausalEvent - Links steps via parent_event_id 6. Write unit tests with >= 90% coverage, including mock MCP server ACCEPTANCE CRITERIA: - Can execute "echo hello" in Docker sandbox and return stdout - Can read a file within workspace and return contents - Denies filesystem access outside workspace - Denies T3/T4 actions - Every action is logged to EventStore with correct causal chain - All tests pass, coverage >= 90%

CODEX PROMPT 6: Module Implementer — State Core

You are the Module Implementer agent for the State Core subsystem of NEXUS-R. CONTEXT: Read specs/04_state_core.spec.md. Phase 1: EventStore (SQLite) + Working State (in-memory). INSTRUCTIONS: Implement modules/state_core/ with: - README.md, src/, tests/, interface.yaml Implementation: 1. event_store.py: Enhanced EventStore (extends foundation version) - Async SQLite with WAL mode for concurrent reads - Methods: append(event), query(filters: dict), get_chain(event_id), get_by_time_range(start, end), get_by_type(event_type), create_projected_view(name, query), compact(older_than_days) - Indexes on: timestamp, event_type, parent_event_id - Schema: events table (id TEXT PK, timestamp TEXT, event_type TEXT, parent_event_id TEXT, data JSON, embedding BLOB) 2. working_state.py: WorkingState class - In-memory dict-based state for current task context - Methods: set(key, value), get(key), get_all(), clear() - Auto-expires when task completes (called by execution loop) 3. identity_store.py: IdentityStore class - Encrypted JSON file for user preferences, tool configs, API key references - Methods: load(), save(), get_preference(key), set_preference(key, value) - Uses Fernet encryption with key derived from user password 4. projected_views.py: ViewManager class - Creates read-optimized views from EventStore - Pre-built views: billing_view (cost aggregation), audit_view (action log), debug_view (causal chains) - Refresh on demand or on new event append 5. Write comprehensive tests with $\geq 90\%$ coverage ACCEPTANCE CRITERIA: - EventStore handles 10,000 events without performance degradation - Causal chain queries return correct ancestry - WorkingState is empty after clear() - IdentityStore round-trips encrypted data - Projected views return correct aggregations - All tests pass, coverage $\geq 90\%$

CODEX PROMPT 7: Module Implementer — Basic Cognition Router (2-Tier)

You are the Module Implementer agent for the Cognition Router subsystem of NEXUS-R. CONTEXT: Read specs/02_cognition_router.spec.md. Phase 1: Basic CAR with 2 tiers only (local via Ollama, BYOK via LiteLLM). CPC short-circuit: query EventStore for matching ETD before routing. INSTRUCTIONS: Implement modules/cognition_router/ with: - README.md, src/, tests/, interface.yaml Implementation: 1. router.py: CognitionRouter class - __init__(config: NEXUSConfig, event_store: EventStore) - route(intent_result: IntentResult) $\rightarrow$ RoutingDecision - RoutingDecision: selected_model (str), selected_tier (T1-T4), cost_estimate (float), etd_match (Optional[ETDEntry]), fallback_chain (list[str]) 2. cpc_check.py: CPCShortCircuit class - check_cache(intent_result: IntentResult) $\rightarrow$ Optional[ETDEntry] - Queries EventStore for ETDs with matching intent_signature - Phase 1: Simple exact-match on intent_type + parameter types - Returns None if no match (triggering model routing) 3. capability_profiler.py: CapabilityProfiler class - Maintains ProviderProfile for available models - Phase 1: Static profiles for local_7b (Ollama) and BYOK (configured in NEXUSConfig) - Method: get_profile(model_id) $\rightarrow$ ProviderProfile - Method: update_from_observation(model_id, task_result) - updates profile based on actual outcomes 4. fallback.py: FallbackChain class - 2-tier chain: local_7b $\rightarrow$ BYOK - Cost cap enforcement at each tier - If local fails verification, escalate to BYOK - Never skip tiers 5. Write tests with $\geq 90\%$ coverage using mocked LiteLLM and Ollama ACCEPTANCE CRITERIA: - route() returns local_7b when privacy_required=True - route() returns BYOK when task_tier $\geq$ T3 - CPC short-circuit returns cached ETD when available - Fallback chain escalates correctly on verification failure - Cost cap is enforced (raises RoutingError if exceeded) - All tests pass, coverage $\geq 90\%$

CODEX PROMPT 8: Module Implementer — Trust Layer (T1-T2)

You are the Module Implementer agent for the Trust Layer of NEXUS-R. CONTEXT: Read specs/06_trust_layer.spec.md. Phase 1: T1-T2 permissions, audit logging, cost tracking, secret handling. INSTRUCTIONS: Implement modules/trust_layer/ with: - README.md, src/, tests/, interface.yaml Implementation: 1. permission_enforcer.py: PermissionEnforcer class - check(action: Action, tier: PermissionTier) → PermissionDecision - T1 allowlist: read files, ls/cat/grep, web search, safe MCP tools - T2 allowlist: write workspace files, sandboxed terminal, HTTP GET to approved domains - Deny-first: any action not on allowlist is denied - Log all decisions to EventStore 2. audit_logger.py: AuditLogger class - log_action(action, result, model_used, tier, cost, user_approved) → audit_id - Query methods: get_audit_trail(task_id), export_audit_log(format, date_range) - Append-only: no delete or modify operations 3. cost_tracker.py: CostTracker class - track(task_id, model_used, tokens_in, tokens_out, cost) → cost_record_id - Query: get_task_cost(task_id), get_total_cost(date_range), get_cost_by_model(date_range) - Store in EventStore as cost events 4. secret_handler.py: SecretHandler class - Uses OS credential manager (keyring library) - Methods: store_secret(key, value), get_secret(key), delete_secret(key), list_keys() - Never log secrets, never include in audit trails, never send to models - Validate BYOK keys on entry (test API call) 5. Write tests with >= 90% coverage ACCEPTANCE CRITERIA: - PermissionEnforcer denies T3/T4 actions in Phase 1 - PermissionEnforcer allows T1 read-only operations - Audit log is append-only (modification raises error) - CostTracker correctly sums costs per task and per date range - SecretHandler never logs secret values - All tests pass, coverage >= 90%

CODEX PROMPT 9: Module Implementer — Workflow Engine (Trace Recording Only)

You are the Module Implementer agent for the Workflow Engine subsystem of NEXUS-R. CONTEXT: Read specs/05_workflow_engine.spec.md. Phase 1: trace recording only. No ETD extraction, distillation, or retrieval yet. INSTRUCTIONS: Implement modules/workflow_engine/ with: - README.md, src/, tests/, interface.yaml Implementation: 1. trace_recorder.py: TraceRecorder class - __init__(event_store: EventStore) - record_step(task_id, step_index, tool, action, input_data, output_data, verification_result, model_used, cost, tier) → event_id - Each step is logged as a CausalEvent with parent_event_id linking to previous step - Method: get_trace(task_id) → list[CausalEvent] (returns ordered execution trace) 2. trace_analyzer.py: TraceAnalyzer class (stub for Phase 2) - analyze_trace(task_id) → TraceAnalysis - Phase 1: Returns basic stats (step_count, total_cost, duration, success_rate) - Placeholder methods for distillation and generalization (to be implemented in Phase 2) 3. Write tests with >= 90% coverage ACCEPTANCE CRITERIA: - TraceRecorder creates correct causal chains (each step links to previous) - get_trace() returns steps in correct order - TraceAnalyzer returns correct basic stats - All events are stored in EventStore - All tests pass, coverage >= 90%

CODEX PROMPT 10: Test Generator — Adversarial Tests for All Phase 1 Modules

You are the Test Generator agent for the NEXUS-R project. Your task is to write adversarial tests, property-based tests, and chaos scenarios for ALL Phase 1 modules. CONTEXT: Read all spec files in specs/ and all implementation code in modules/. You are the second pair of eyes that validates every module. INSTRUCTIONS: Create the following test files in tests/: 1.

tests/unit/test_input_gateway_adversarial.py: - Property: For any input, parse() always returns a valid IntentResult (never crashes) - Adversarial: empty string, very long string (>10000 chars), unicode, SQL injection, prompt injection - Chaos: random byte sequences, rapidly changing inputs 2.

tests/unit/test_execution_sandbox_adversarial.py: - Property: No action outside workspace is ever permitted - Adversarial: path traversal (../../etc/passwd), command injection (; rm -rf /), symlink attacks - Chaos: kill sandbox process mid-execution, fill disk, exceed memory 3.

tests/unit/test_cognition_router_adversarial.py: - Property: Privacy requirement always forces local model selection - Adversarial: conflicting requirements (high complexity + privacy), zero cost cap, all models unavailable - Chaos: random model profiles, rapidly changing availability 4.

tests/unit/test_state_core_adversarial.py: - Property: EventStore is always append-only (no modification or deletion) - Adversarial: concurrent writes, corrupt SQLite, extremely large events, missing parent_event_id - Chaos: disk full during write, process kill during transaction 5.

tests/unit/test_trust_layer_adversarial.py: - Property: No T3/T4 action is ever auto-approved in Phase 1 - Adversarial: action type spoofing, privilege escalation attempts, secret exfiltration via audit log - Chaos: concurrent permission checks, rapid action submission 6.

tests/unit/test_workflow_engine_adversarial.py: - Property: Every recorded step has valid parent_event_id chain - Adversarial: circular references, missing steps, out-of-order timestamps - Chaos: concurrent trace recording, process kill during recording

ACCEPTANCE CRITERIA: - All adversarial tests pass (system handles all attacks gracefully) - All property-based tests pass (invariants hold for random inputs) - Total test count >= 200 across all files - No test passes by catching unexpected exceptions (each failure mode must be explicitly handled) - Coverage of each module >= 90% when combined with module's own tests

CODEX PROMPT 11: Integration Validator — Cross-Module Integration Tests

You are the Integration Validator agent for the NEXUS-R project. Your task is to verify that all Phase 1 modules work together correctly. CONTEXT: Read all spec files in specs/ and all module interface.yaml files. You must verify cross-module contracts. INSTRUCTIONS: Create the following integration test files: 1. tests/integration/test_full_pipeline.py: - End-to-end: user input → Input Gateway → Cognition Router → Execution Sandbox → State Core → Trust Layer - Test: "list all python files in workspace" → T1 task → local model → sandboxed ls → event logged → cost tracked - Test: "create a new file test.txt with hello world" → T2 task → local model → sandboxed write → permission check → event logged - Test: "delete system files" → T4 task → DENIED by Trust Layer → audit logged 2. tests/integration/test_event_sourcing.py: - Verify: Every action across all modules produces an EventStore entry - Verify: Causal chains are consistent across modules - Verify: Projected views (billing, audit, debug) reflect actual execution - Verify: EventStore can reconstruct full task history from events alone 3. tests/integration/test_routing_with_cpc.py: - Test: Execute same task twice → second time uses CPC short-circuit - Verify: Cost of second execution is \$0 (local ETD) or lower than first - Verify: ETD match is logged in routing decision 4. tests/integration/test_permission_enforcement.py: - Test: T1 task cannot trigger T2 actions without approval - Test: T2 task cannot trigger T3+ actions in Phase 1 - Verify: Every denied action is logged in audit trail 5. tests/integration/test_cost_tracking.py: - Execute multiple tasks and verify cumulative cost tracking - Verify cost breakdown by model, by tier, by task type - Verify cost cap enforcement (task stops when cap is reached) ACCEPTANCE CRITERIA: - All integration tests pass - Full pipeline test executes in < 30 seconds - Event sourcing test confirms 100% action coverage - CPC short-circuit test shows measurable cost reduction - No integration test requires external API calls (all mocked)

CODEX PROMPT 12: Security Auditor — Phase 1 Security Review

You are the Security Auditor agent for the NEXUS-R project. Your task is to review ALL Phase 1 code for security vulnerabilities. CONTEXT: Read all implementation code and specs. Focus on: prompt injection, sandbox escapes, secret leaks, privilege escalation, and audit log integrity. INSTRUCTIONS: 1. Review all modules for injection vectors: - Command injection in Execution Sandbox terminal commands - Path traversal in filesystem operations - Prompt injection in Input Gateway (user input reaching model without framing) - SQL injection in EventStore queries 2. Review sandbox isolation: - Docker configuration: no privileged mode, no host network, minimal capabilities - Filesystem scoping: verify workspace boundary enforcement - MCP client: verify no arbitrary tool creation 3. Review secret handling: - Verify secrets never appear in logs, audit trails, or ETD records - Verify secrets are never sent to models - Verify BYOK key validation on entry 4. Review audit log integrity: - Verify append-only enforcement at code level - Verify no audit log modification or deletion paths exist 5. Write a security audit report: security_audit_phase1.md in docs/ - For each finding: severity (Critical/High/Medium/Low), description, affected code, recommended fix - Implement fixes for all Critical and High findings - Create tests/security/test_security_hardening.py for all fixes ACCEPTANCE CRITERIA: - Zero Critical or High severity findings remaining after fixes - All security tests pass - Security audit report is comprehensive (>3000 words) - No hardcoded secrets in codebase - Sandbox cannot be escaped in any test scenario

CODEX PROMPT 13: CLI Interface + End-to-End Wiring

You are the Module Implementer agent for the NEXUS-R CLI interface. Your task is to wire all Phase 1 modules together behind a functional CLI. CONTEXT: All Phase 1 modules are implemented and tested. You need to create the user-facing CLI and the main execution loop. INSTRUCTIONS: 1. Create modules/cli/ with: - src/main.py: Main CLI using Typer or Click - Commands: nexus run "", nexus history, nexus cost, nexus config 2. Create modules/orchestrator/ with: - src/orchestrator.py: MainOrchestrator class - Orchestrates the full pipeline: Input Gateway → Cognition Router → Execution Sandbox - Manages Working State lifecycle (create on task start, clear on task end) - Logs all orchestration decisions to EventStore - Handles errors with user-friendly messages 3. Wire the execution loop: - Accept user input → parse intent → check CPC → route model → execute in sandbox → verify each step → log events → return result - On CPC hit: skip routing, execute cached workflow - On verification failure: escalate through fallback chain - On permission denial: prompt user (T3) or deny (T4 in Phase 1) 4. Add cost display: after each task, show model used, tokens consumed, cost, and tier 5. Write e2e/ tests that run the full CLI with real (mocked) backends ACCEPTANCE CRITERIA: - `nexus run "list all python files"` executes and returns results - `nexus run "create test.txt"` creates the file and shows cost - `nexus history` shows recent task executions - `nexus cost` shows cumulative cost breakdown - `nexus config` shows current configuration - End-to-end tests pass for all CLI commands - Full pipeline execution for T1 task completes in < 5 seconds

10.2 Phase 2: ETD + Full CAR (Weeks 9–16)

Goal: Does ETD measurably reduce cost and latency on repeated tasks? Do users feel in control? This phase implements the core innovation (ETD), expands the Cognition Router to 4 tiers, adds a cost dashboard UI, and extends the Trust Layer to T3–T4 permissions. Validation: A/B test with 10 users using ETD enabled vs. 10 without; measure cost reduction and latency improvement.

CODEX PROMPT 14: Spec Architect — Phase 2 Specification Updates

You are the Spec Architect agent for NEXUS-R Phase 2. Update specifications to cover ETD, full CAR, T3-T4 permissions, and cost dashboard. INSTRUCTIONS: 1. Update specs/02_cognition_router.spec.md: - Full 4-tier CAR: local_7b, local_14b, local_70b, BYOK_budget, BYOK_frontier, managed_premium - Adaptive fallback chain with cost caps at each tier - Parallel probe for uncertain tasks (send to T1 and T2 simultaneously) - De-escalation learning (update classifier when T1 succeeds for T2-classified tasks) - Define new error codes CR-051 to CR-100 2. Update specs/05_workflow_engine.spec.md: - Full ETD pipeline: distillation, generalization verification (>90% threshold), parameterization, deduplication, retrieval, conservative application, invalidation - Define ETD entry schema (JSON), distillation algorithm, generalization test procedure - Define retrieval ranking algorithm: $\text{success_rate} * \text{recency_weight}$ - Define invalidation triggers: $\text{failure_rate} > 0.3$, dependency version change, 180 days unused - Define new error codes WE-031 to WE-100 3. Update specs/06_trust_layer.spec.md: - T3 permissions: arbitrary terminal, files outside workspace, HTTP POST/PUT/DELETE, browser actions, API keys - T4 permissions: delete files, access secrets, deploy to prod, financial transactions - ML risk classifier: $\text{risk_score} = f(\text{action_type}, \text{target_resource}, \text{permission_scope}, \text{user_history})$ - Define risk scoring algorithm, thresholds (low/medium/high), and escalation rules - Define new error codes TL-051 to TL-100 4. Create specs/07_cost_dashboard.spec.md: - Web UI for cost visualization: per-task cost, cumulative cost, cost by model, cost by tier - Read-only audit log view - Real-time cost tracking during task execution - Define page layouts, data queries, and API endpoints ACCEPTANCE CRITERIA: - All updated specs maintain backward compatibility with Phase 1 - New error codes do not overlap with Phase 1 codes - ETD pipeline has complete algorithm descriptions (not just interface definitions) - ML risk classifier spec includes training data requirements and evaluation metrics

CODEX PROMPT 15: ETD Distiller — Implement Workflow Engine ETD Pipeline

You are the ETD Distiller agent for the NEXUS-R Workflow Engine. Your task is to implement the full ETD pipeline. CONTEXT: Read specs/05_workflow_engine.spec.md (Phase 2 version). The Phase 1 TraceRecorder and TraceAnalyzer are already implemented. INSTRUCTIONS: Implement in modules/workflow_engine/src/: 1. distiller.py: TraceDistiller class - distill(trace: list[CausalEvent]) → DistilledWorkflow - Extract minimal causal chain: drop dead-ends, retries, exploratory steps - Identify task-specific vs. task-agnostic parameters - Mark decision points with alternatives 2. generalizer.py: GeneralizationVerifier class - verify(workflow: DistilledWorkflow, test_variants: list[Task]) → GeneralizationResult - Test distilled workflow on held-out task variants - Measure success rate across variations - Return GeneralizationResult: success_rate, tested_variants, failure_cases - Only admit workflows with >90% generalization success 3. parameterizer.py: WorkflowParameterizer class - parameterize(workflow: DistilledWorkflow) → ETD - Replace concrete values with typed parameter slots - Define input_schema, output_schema, parameter_slots - Generate intent_signature and compute intent_embedding 4. retriever.py: ETDRetriever class - retrieve(intent: IntentResult) → Optional[ETD] - Cosine similarity matching on intent_embeddings - Context gating: check invariant_checks against current environment - Ranking: success_rate * recency_weight - Hard threshold: similarity > 85% AND success_rate > 90% 5. applicator.py: ConservativeApplicator class - apply(etd: ETD, parameters: dict) → ExecutionPlan - Verify environment match (invariant checks) - If any step fails verification, abandon ETD and return None - Validate output schema after execution - Update success/failure counts 6. invalidator.py: ETDInvalidator class - check_invalidations(etd: ETD) → bool - Triggers: failure_rate > 0.3, invalidation triggers fired, 180 days unused - Mark degraded (not deleted) - EventStore is append-only 7. Write comprehensive tests with >= 90% coverage ACCEPTANCE CRITERIA: - Distiller correctly extracts minimal causal chains from traces - GeneralizationVerifier rejects workflows with <90% success rate - Parameterizer generates valid ETD entries with correct schemas - Retriever returns matching ETDs above similarity threshold - Applicator abandons failing ETDs and falls back correctly - Invalidator correctly identifies degraded ETDs - All tests pass, coverage >= 90%

CODEX PROMPT 16: Module Implementer — Full 4-Tier Cognition Router

You are the Module Implementer agent for the Cognition Router. Upgrade from 2-tier to full 4-tier CAR. CONTEXT: Read updated specs/02_cognition_router.spec.md. Phase 1 basic router is implemented. INSTRUCTIONS: Update modules/cognition_router/: 1. router.py: Upgrade CognitionRouter - 6-tier model chain: local_7b → local_14b → local_70b → BYOK_budget → BYOK_frontier → managed_premium - Never jump more than one tier at a time - User gate at managed_premium (always requires explicit approval) - Cost caps at each tier 2. parallel_probe.py: ParallelProber class - probe(task: TaskDefinition, tier1: str, tier2: str) → ProbeResult - Send task to both tiers simultaneously - If tier1 succeeds verification → discard tier2 result - Return whichever succeeds first 3. de_escalation.py: DeEscalationLearner class - learn(task_signature: str, assigned_tier: int, actual_tier: int) - If task consistently succeeds at lower tier than classified, update classifier weights - Persist learning in IdentityStore 4. capability_profiler.py: Upgrade with dynamic profiles - Update profiles based on observed outcomes (feedback loop) - Track per-model success rates, latency distributions, cost accuracy 5. Update all tests for new functionality ACCEPTANCE CRITERIA: - 4-tier routing selects correct tier based on task complexity - Parallel probe correctly discards higher-tier result when lower succeeds - De-escalation learning reduces routing tier over repeated similar tasks - Cost cap enforcement works at all tiers - managed_premium requires user approval - All tests pass, coverage >= 90%

CODEX PROMPT 17: Module Implementer — Trust Layer T3-T4 + ML Risk Classifier

You are the Module Implementer agent for the Trust Layer. Upgrade to T3-T4 permissions and add ML risk classification. CONTEXT: Read updated specs/06_trust_layer.spec.md. Phase 1 T1-T2 implementation exists. INSTRUCTIONS: Update modules/trust_layer/: 1. permission_enforcer.py: Add T3 and T4 permissions - T3: User prompt required (arbitrary terminal, external files, HTTP POST/PUT/DELETE, browser actions, API keys) - T4: Explicit confirmation with documented reason (delete files, access secrets, deploy, financial transactions) - Implement user interaction for T3/T4 approval via CLI prompts 2. risk_classifier.py: MLRiskClassifier class - classify_risk(action: Action, target: str, scope: str, user_history: list) → RiskScore - Phase 2: Rule-based classifier (no ML model yet) - Rules: action_type risk weights, target_resource sensitivity, permission_scope escalation, user_history patterns - Thresholds: low_risk → auto, medium_risk → prompt user, high_risk → explicit override + audit - Runs locally, target < 10ms per classification 3. prompt_injection_defense.py: PromptInjectionDetector class - detect(user_input: str) → InjectionAssessment - Input framing: wrap external content in delimiters - Anomaly detection: T4 requests during T1 task → flag and pause - Output validation: parse structurally, never eval() 4. Update all tests ACCEPTANCE CRITERIA: - T3 actions require user prompt - T4 actions require explicit confirmation with reason - Risk classifier runs in < 10ms per action - Prompt injection defense catches > 90% of test injection attempts - No T4 action is auto-approved - All tests pass, coverage >= 90%

CODEX PROMPT 18: Cost Dashboard Web UI

You are the Module Implementer agent for the Cost Dashboard. Build a web UI for cost visualization and audit log viewing. CONTEXT: Read specs/07_cost_dashboard.spec.md. The State Core provides projected views for billing and audit data. INSTRUCTIONS: Create modules/web_ui/ with: - README.md, src/, tests/, interface.yaml Implementation: 1. app.py: FastAPI application - Endpoints: GET /api/costs (per-task + cumulative), GET /api/costs/by-model, GET /api/costs/by-tier - Endpoints: GET /api/audit (paginated audit log), GET /api/audit/{task_id} - Endpoints: GET /api/etd (list ETDs with stats), GET /api/etd/{id} - WebSocket: /ws/costs (real-time cost updates during task execution) 2. static/: Simple HTML/CSS/JS dashboard - Cost overview page: total cost, cost trend chart, cost by model pie chart, cost by tier bar chart - Audit log page: paginated table with filtering and search - ETD management page: list of ETDs with success rates, invalidation status - Use Chart.js for visualizations 3. Write integration tests ACCEPTANCE CRITERIA: - Dashboard displays correct cost data from EventStore - Audit log is searchable and paginated - ETD list shows all cached workflows with stats - Real-time cost updates work via WebSocket - UI is responsive and functional - All API endpoints return correct data

CODEX PROMPT 19: Integration Validator — Phase 2 Integration Tests

You are the Integration Validator agent for NEXUS-R Phase 2. Verify ETD pipeline and full CAR work end-to-end. INSTRUCTIONS: Create tests/integration/phase2/: 1. test_etd_lifecycle.py: Full ETD lifecycle - Execute a task → record trace → distill → verify generalization → parameterize → store ETD → retrieve for similar task → apply → validate output - Verify: ETD reduces cost on second execution of similar task - Verify: Generalization verification rejects workflows below 90% threshold 2. test_full_car_routing.py: 4-tier routing scenarios - T1 task routes to local_7b - T4 task routes to BYOK_frontier or managed_premium (with approval) - Parallel probe: uncertain task sent to two tiers - Fallback: T1 fails → escalate to T2 → succeed 3. test_t3_t4_permissions.py: Permission enforcement - T3 action prompts user - T4 action requires explicit confirmation + reason - Prompt injection attempt is caught and blocked 4. test_cost_dashboard_integration.py: Dashboard data accuracy - Execute tasks → verify dashboard shows correct costs - Verify audit log matches actual executions - Verify ETD stats are accurate 5. test_de_escalation.py: Classifier learning - Execute same task type 10 times → verify routing tier decreases ACCEPTANCE CRITERIA: - ETD reduces cost by >40% on repeated similar tasks - Full CAR correctly routes to all 6 model tiers - T3/T4 permissions are enforced without exception - Dashboard data matches EventStore records - De-escalation learning reduces routing tier over iterations

CODEX PROMPT 20: Security Auditor — Phase 2 Security Review

You are the Security Auditor agent for NEXUS-R Phase 2. Review all new code for security vulnerabilities. INSTRUCTIONS: 1. Review ETD pipeline for data leakage: - Verify secrets never appear in ETD entries - Verify ETD entries cannot be used to extract user data - Verify generalization testing does not expose execution details 2. Review ML risk classifier for adversarial inputs: - Test risk classifier with crafted inputs designed to produce false low-risk scores - Verify classifier cannot be bypassed by manipulating action metadata 3. Review web UI for security: - Verify API endpoints require authentication - Verify WebSocket connections are authenticated - Verify no XSS in dashboard rendering - Verify no SQL injection in audit log queries 4. Review T3/T4 permission prompts: - Verify T4 confirmation cannot be bypassed programmatically - Verify audit log captures all T3/T4 decisions 5. Write security_audit_phase2.md and implement fixes 6. Create tests/security/test_phase2_hardening.py ACCEPTANCE CRITERIA: - Zero Critical or High findings remaining - All security tests pass - Web UI requires authentication for all endpoints - ETD entries contain no secrets or sensitive user data - Risk classifier is not bypassable by input manipulation

10.3 Phase 3: Browser + Polish (Weeks 17–24)

Goal: Full end-to-end capability: terminal + files + browser + APIs. ETD composition (chaining multiple ETDs). Three-tier memory system fully operational. User preferences and personalization. Execution replay and fork from event log. Polished Web UI. Managed premium tier integration. This phase transforms NEXUS-R from a CLI tool into a complete agent runtime.

CODEX PROMPT 21: Spec Architect — Phase 3 Specification Updates

You are the Spec Architect for NEXUS-R Phase 3. Add browser automation, ETD composition, full 3-tier memory, execution replay, and user personalization. INSTRUCTIONS: Update specs/: 1. Update specs/03_execution_sandbox.spec.md: Add browser automation via MCP (Playwright), isolated Chromium, API call tools 2. Update specs/04_state_core.spec.md: Full 3-tier memory with vector embeddings, Deep Memory compaction 3. Update specs/05_workflow_engine.spec.md: ETD composition (chain 2+ ETDs), execution replay, fork from event log 4. Create specs/08_identity_personalization.spec.md: User preferences, behavioral policies, classifier weights, tool configs 5. Create specs/09_web_ui_polish.spec.md: Polished UI design, real-time updates, dark mode, mobile-responsive ACCEPTANCE CRITERIA: All new specs are complete with function signatures, error codes, and test scenarios.

CODEX PROMPT 22: Module Implementer — Browser Automation via MCP

You are the Module Implementer for the Execution Sandbox. Add browser automation capabilities. INSTRUCTIONS: Update modules/execution_sandbox/: 1. browser_sandbox.py: BrowserSandbox class using Playwright via MCP - Isolated Chromium instance, no host filesystem access, no real browser profile - Methods: navigate(url), click(selector), type(selector, text), screenshot(), extract(selector) - All actions logged as CausalEvents in EventStore - Permission check: browser actions require T3 approval 2. api_tools.py: APITools class for HTTP requests - GET, POST, PUT, DELETE with domain whitelisting - Credential injection via SecretRegistry (never expose to model) - T2: GET to approved domains, T3: all methods + arbitrary domains 3. Update sandbox.py to orchestrate terminal + filesystem + browser + API tools 4. Write comprehensive tests ACCEPTANCE CRITERIA: - Browser can navigate to a URL and extract text - Browser cannot access host filesystem - API tools enforce domain whitelisting - All actions are event-sourced - Browser actions require T3 permission - Coverage $\geq 90\%$

CODEX PROMPT 23: Module Implementer — ETD Composition and Execution Replay

You are the ETD Distiller agent. Implement ETD composition and execution replay. INSTRUCTIONS: Update modules/workflow_engine/: 1. composer.py: ETDComposer class - compose(etd_ids: list[str]) $\rightarrow$ CompositeETD - Chain multiple ETDs: output of ETD1 feeds into input of ETD2 - Validate: output_schema of ETD1 matches input_schema of ETD2 - Track composition as new ETD with derived_from_events 2. replayer.py: ExecutionReplayer class - replay(task_id: str) $\rightarrow$ ReplayResult - Reconstruct execution from EventStore causal chain - Display step-by-step replay with timestamps, costs, and decisions 3. forker.py: ExecutionForker class - fork(task_id: str, from_step: int) $\rightarrow$ ForkResult - Create new execution starting from a specific step in a past execution - Copy Working State from fork point, create new causal chain 4. Write tests for all new functionality ACCEPTANCE CRITERIA: - Composer chains 2+ ETDs correctly with schema validation - Replayer produces step-by-step execution replay - Forker creates valid fork from any step in past execution - Composition produces new ETD with correct provenance - Coverage $\geq 90\%$

CODEX PROMPT 24: Module Implementer — Full Three-Tier Memory with Vector Embeddings

You are the Module Implementer for State Core. Implement full 3-tier memory with vector embeddings and Deep Memory compaction. INSTRUCTIONS: Update modules/state_core/: 1. vector_index.py: VectorIndex class - Index event embeddings using FAISS or similar - Methods: index(event_id, embedding), search(query_embedding, k) → list[(event_id, score)] - Update index on new event append 2. memory_compactor.py: DeepMemoryCompactor class - compact(events: list[Event]) → list[CompactEvent] - Semantic compression: merge similar events, extract summaries - Target: 71% token reduction (as demonstrated in Deep Memory research) - Preserve causal chains in compacted representation 3. Update working_state.py: Add conversation history management 4. Update identity_store.py: Add behavioral policies, classifier weights, tool configurations 5. Write tests ACCEPTANCE CRITERIA: - Vector search returns semantically similar events - Compaction achieves >60% token reduction while preserving causal chains - Working State correctly manages conversation history - Identity Store persists behavioral policies across sessions - Coverage >= 90%

CODEX PROMPT 25: Module Implementer — User Personalization and Identity

You are the Module Implementer for the Identity and Personalization subsystem. INSTRUCTIONS: Create modules/personalization/ (or extend modules/state_core/identity_store.py): 1. preference_learner.py: PreferenceLearner class - Learn user preferences from execution history - Methods: learn_from_execution(task, result), get_preferred_model(task_type), get_preferred_tools(task_type) - Store preferences in IdentityStore 2. policy_engine.py: PolicyEngine class - User-defined behavioral policies: always_local, never_cloud, cost_cap_per_day, auto_approve_t2, etc. - Evaluate policies before each execution - Override routing decisions based on policies 3. classifier_trainer.py: ClassifierTrainer class - Fine-tune task classifier on user's execution history - Update classifier weights in IdentityStore - Phase 3: Simple logistic regression on extracted features 4. Write tests ACCEPTANCE CRITERIA: - Preference learner correctly identifies user model preferences - Policy engine enforces user policies without exception - Classifier trainer improves classification accuracy over time - All preferences and policies persist in IdentityStore - Coverage >= 90%

CODEX PROMPT 26: Polished Web UI

You are the Module Implementer for the Web UI. Polish the cost dashboard into a full-featured interface. INSTRUCTIONS: Update modules/web_ui/: 1. Add pages: Dashboard home, Task history, ETD management, Settings, Audit log 2. Real-time task execution monitoring (WebSocket) 3. Dark mode toggle 4. Mobile-responsive design 5. Task submission from web UI (in addition to CLI) 6. Execution replay visualization (step-by-step with timeline) 7. ETD composition builder (drag-and-drop workflow chaining) 8. User preferences editor ACCEPTANCE CRITERIA: - All pages load and display correct data - Real-time updates work via WebSocket - Dark mode toggles correctly - Mobile layout is functional - Task submission works from web UI - Execution replay is interactive - ETD composer validates schemas before chaining

CODEX PROMPT 27: Integration Validator — Phase 3 Integration Tests

You are the Integration Validator for Phase 3. Test browser automation, ETD composition, full memory, and personalization end-to-end. INSTRUCTIONS: Create tests/integration/phase3/: 1. test_browser_pipeline.py: Full browser automation pipeline 2. test_etd_composition.py: Chain 2+ ETDs end-to-end 3. test_execution_replay.py: Replay and fork past executions 4. test_personalization.py: User preferences affect routing decisions 5. test_full_memory.py: 3-tier memory with vector search and compaction 6. test_web_ui_e2e.py: Full web UI interaction tests ACCEPTANCE CRITERIA: All Phase 3 integration tests pass. ETD composition reduces cost by >60% on multi-step workflows. Browser automation succeeds on >80% of test scenarios. Personalization measurably improves routing accuracy over baseline.

CODEX PROMPT 28: Security Auditor — Phase 3 Security Review

You are the Security Auditor for Phase 3. Review browser automation, web UI, ETD composition, and personalization. INSTRUCTIONS: 1. Review browser sandbox isolation (Chromium security, credential leakage) 2. Review web UI authentication and authorization 3. Review ETD composition for privilege escalation (chaining T1 + T4 ETDs) 4. Review personalization data privacy (preferences, policies, classifier weights) 5. Write security_audit_phase3.md and implement fixes 6. Create tests/security/test_phase3_hardening.py ACCEPTANCE CRITERIA: Zero Critical or High findings. Browser sandbox cannot access host system. Web UI is fully authenticated. ETD composition respects permission boundaries. Personalization data is encrypted at rest.

10.4 Phase 4: Community + Scale (Weeks 25+)

Goal: Community ETD exchange, multi-provider premium tier, advanced CAR, third-party API, and mobile companion. This phase transforms NEXUS-R from a personal tool into a community ecosystem. The ETD community exchange creates network effects that are difficult for competitors to replicate: users who share workflows benefit from the collective intelligence of the community, and switching away means losing access to that accumulated knowledge. The mobile companion enables users to monitor costs, review audit logs, and approve escalations from their phone, making the system practical for daily use outside the terminal.

CODEX PROMPT 29: Spec Architect — Phase 4 Specification Updates

You are the Spec Architect for Phase 4. Spec the community ETD exchange, advanced CAR, and mobile companion. INSTRUCTIONS: Create/update specs/: 1. Create specs/10_etd_community.spec.md: Opt-in sharing, privacy stripping, verification pipeline, revenue share model 2. Update specs/02_cognition_router.spec.md: Advanced CAR with learned classifier (fine-tuned model) 3. Create specs/11_mobile_companion.spec.md: View audit, approve escalations, cost alerts 4. Create specs/12_third_party_api.spec.md: REST API for integrations, rate limiting, authentication ACCEPTANCE CRITERIA: All Phase 4 specs are complete with function signatures, error codes, and test scenarios.

CODEX PROMPT 30: Module Implementer — ETD Community Exchange

You are the Module Implementer for the ETD Community Exchange. INSTRUCTIONS: Create modules/etd_exchange/: 1. exchange_client.py: ExchangeClient class - upload_etd(etd: ETD) → share_id (opt-in, privacy-stripped) - download_etd(share_id) → ETD - search_etd(intent: str) → list[ETD] - Rate limit: 100 uploads/day, 1000 downloads/day 2. privacy_stripper.py: PrivacyStripper class - strip(etd: ETD) → StrippedETD (remove all user-specific data, paths, secrets, personal patterns) - Verify: no file paths, no API keys, no user-specific values remain - Generalize: replace all user-specific patterns with generic placeholders 3. verification_pipeline.py: CommunityVerification class - verify_uploaded(etd: StrippedETD) → VerificationResult - Run generalization tests on community ETDs before making available - Require >80% generalization success for community ETDs (lower than personal threshold) 4. Write tests ACCEPTANCE CRITERIA: - Upload/download cycle preserves ETD structure - Privacy stripper removes all user-specific data - Community ETDs are verified before availability - Rate limiting works correctly - Coverage >= 90%

CODEX PROMPT 31: Module Implementer — Advanced CAR with Learned Classifier

You are the Module Implementer for the advanced Cognition Router with a learned classifier. INSTRUCTIONS: Update modules/cognition_router/: 1. learned_classifier.py: LearnedTaskClassifier class - Replace rule-based classifier with fine-tuned small model - Training data: accumulated execution history from EventStore - Features: task text embedding, complexity scores, historical success rates - Output: task tier (T1-T4) with confidence 2. adaptive_profiler.py: AdaptiveCapabilityProfiler class - Continuously update model profiles based on observed outcomes - Track per-model accuracy by task type - Adjust routing weights based on performance trends 3. Write tests with synthetic training data ACCEPTANCE CRITERIA: - Learned classifier achieves >85% accuracy on held-out test data - Classifier improves over time with more execution data - Adaptive profiler correctly adjusts to model performance changes - Routing decisions are explainable (which features contributed) - Coverage >= 85%

CODEX PROMPT 32: Module Implementer — Third-Party API + Mobile Companion Backend

You are the Module Implementer for the third-party API and mobile companion backend. INSTRUCTIONS: Create modules/api_gateway/: 1. rest_api.py: FastAPI application - POST /api/v1/tasks: Submit task for execution - GET /api/v1/tasks/{id}: Get task status and result - GET /api/v1/costs: Get cost summary - GET /api/v1/audit: Get audit trail - POST /api/v1/approvals/{id}: Approve/deny escalation - WebSocket /api/v1/ws: Real-time task updates - API key authentication, rate limiting (1000 req/hour) 2. mobile_api.py: Mobile-specific endpoints - GET /api/v1/mobile/dashboard: Summary for mobile - POST /api/v1/mobile/approve: Quick approval from mobile - GET /api/v1/mobile/costs: Cost alerts and summaries - Push notification support (Firebase Cloud Messaging) 3. Write tests ACCEPTANCE CRITERIA: - REST API is fully functional and documented (OpenAPI/Swagger) - API key authentication works - Rate limiting prevents abuse - Mobile endpoints return appropriate data - WebSocket provides real-time updates - Coverage >= 90%

CODEX PROMPT 33: Final Integration — Full System End-to-End Validation

You are the Integration Validator for the complete NEXUS-R system. Run comprehensive end-to-end validation across all phases. INSTRUCTIONS: Create tests/e2e/full_system/: 1. test_complete_lifecycle.py: Full task lifecycle across all subsystems 2. test_etd_accumulation.py: Execute 100 tasks, verify ETD accumulation reduces cost progressively 3. test_community_exchange.py: Upload, download, and apply community ETDs 4. test_mobile_approval.py: Submit T3/T4 task, approve from mobile API 5. test_security_regression.py: Run all security tests from all phases 6. test_performance.py: Verify routing overhead <50ms, E2E latency <3s for T1 Run ALL tests across ALL phases and generate a comprehensive test report. ACCEPTANCE CRITERIA: - All end-to-end tests pass - ETD accumulation shows >60% cost reduction after 50+ tasks - Community ETDs are verified and applicable - Mobile approval workflow completes in <10 seconds - No security regressions from Phase 1-3 - Routing overhead <50ms - T1 task E2E latency <3s - Full test report generated with pass/fail rates per module and per phase

11. Counterarguments and Limitations

11.1 Risk Register

The following risk register identifies the most significant threats to the NEXUS-R project, synthesized from both predecessor documents with honest severity and likelihood assessments. Each risk includes a mitigation strategy that has been validated against the architectural decisions described in this plan. The register is ordered by risk priority (severity * likelihood), not by when the risk might materialize.

Risk	Severity	Likelihood	Mitigation
ETD does not generalize (workflows too variable)	High	Medium	Start narrow (deploy, test, data pipeline). Measure empirically. Kill if <70% success rate.
Scope creep kills focus	High	High	Phase 1 = CLI + terminal + files ONLY. No browser, no APIs, no premium. Strict phase gates.
Big players build this (OpenAI, Anthropic)	High	High	Ship fast. ETD community creates network effects they cannot replicate. First-mover advantage in workflow knowledge accumulation.
Local models too weak for useful execution	Medium	Medium	Qwen 2.5 7B / Phi-4 already competent for simple tasks. CAR handles the gap via adaptive fallback.
Users do not trust local agents with file/terminal access	Medium	Medium	Strong permissions + audit + rollback. Start read-only. Demonstrate value before requesting elevated permissions.
Security breach in sandbox	High	Low	Docker isolation + deny-first + ML risk classifier. Bug bounty program after Phase 2.

BYOK key compromise	Medium	Low	OS credential manager + per-session isolation + audit logging. Key rotation support.
Routing overhead adds latency	Low	Low	Target <50ms. Research confirms routing is cheap vs. LLM inference (2-15s).

Table 8: Risk Register — Synthesized from Both Predecessor Documents

11.2 ETD Generalization Concerns

The most substantive criticism of the ETD approach is that workflows may be too variable to generalize effectively. If each execution of a "deploy to Vercel" task differs significantly in its step sequence (due to project-specific build configurations, environment variables, or dependency issues), then the distillation process may produce workflows that succeed on the original task variants but fail on new ones—or worse, succeed superficially but produce incorrect results. The v7 specification addressed this concern by introducing explicit generalization verification with a >90% success rate threshold, but the question remains: is 90% sufficient for user trust? A 1-in-10 failure rate on cached workflows may be acceptable for low-stakes tasks (formatting code, searching documentation) but unacceptable for high-stakes tasks (deploying to production, executing financial transactions).

The mitigation is threefold. First, the conservative application strategy ensures that ETD failures are caught early by inline verification at each step, with immediate fallback to fresh reasoning. The user never sees a "wrong answer" from an ETD—they see at worst a brief delay while the system falls back. Second, ETD reliability is task-type-dependent: the system should only cache workflows for task types where generalization has been empirically validated, starting with narrow, well-structured domains (deployment, testing, data pipelines) and expanding only as evidence accumulates. Third, the invalidation mechanism ensures that degraded ETDs are removed from circulation, preventing accumulated failures from eroding trust. If the generalization success rate for a task type falls below 70%, the ETD for that task type should be killed entirely—this is the explicit kill criterion stated in the v8 specification.

11.3 Local Model Capability Gaps

A pragmatic limitation of the local-first approach is that current local models (even 70B parameter models running on consumer hardware) may not match the capability of frontier cloud models for complex reasoning tasks. The adaptive fallback chain addresses this by escalating to more capable models when local execution fails verification, but the cost of failed local attempts (in terms of latency, not money) may degrade user experience. The parallel probe mechanism mitigates this by sending uncertain tasks to both local and BYOK models simultaneously, but this introduces additional computational overhead. The honest assessment is that local models will handle 80% of common tasks effectively (as stated in the performance targets), but the remaining 20% will require cloud models, and the transition between local and cloud must be seamless to avoid user frustration. The de-escalation learning mechanism is

designed to continuously improve the proportion of tasks that can be handled locally, but this improvement is gradual and cannot be guaranteed to reach any specific threshold within a given timeframe.

11.4 Competitive Threat Analysis

The most significant competitive threat is that major AI providers (OpenAI, Anthropic, Google) could build similar workflow caching mechanisms into their existing agent products. Claude Code already offers terminal-first deep reasoning; OpenAI Codex provides cloud-based autonomous coding; Cursor and Windsurf offer IDE-native AI assistance. None of these currently implement verified workflow reuse, but the technical barrier to entry is not insurmountable. The defense is two-fold. First, NEXUS-R's ETD community exchange creates network effects that are difficult for a single-provider system to replicate: users share workflows across different model providers, creating a provider-agnostic knowledge base that no single provider can own. Second, NEXUS-R's local-first, BYOK architecture means that users are not locked into any single provider's ecosystem—their accumulated ETDs work with any model, and the cost savings compound regardless of which provider they use for non-cached tasks. The strategic implication is clear: ship fast, build the community early, and make ETD accumulation the switching cost that competitors cannot overcome.

12. Conclusion and Implications

This document has presented a unified master plan for the NEXUS-R project, synthesizing the architectural specification (v8.0) with the agent-native development methodology into a single, comprehensive document organized as a sequence of 33 copy-paste Codex prompts. The synthesis resolves the critical gap between what to build and how to build it, providing a complete roadmap from initial repository setup through community-scale deployment. The plan is grounded in the research contributions of OpenHands V1 [1], MoMA [2], FlowMind [3], and Shepherd [4], and it adheres to the inviolable principles of specification-first development, agent isolation, and verification-before-integration.

The central innovation—Execution Trace Distillation with verified generalization bounds—addresses a fundamental economic problem in AI agent systems: the compounding cost of repetitive task execution. By caching verified, parameterized workflows with quantified reliability guarantees (>90% generalization success rate) and applying them conservatively (similarity >85%, opt-in, immediate fallback on failure), NEXUS-R creates a system where every successful execution makes the next one faster and cheaper. This is not a theoretical claim—it is an engineering proposition that can be empirically validated in Phase 2: if ETD does not demonstrate >40% cost reduction on repeated tasks, the project should be killed. This kill criterion is explicit and non-negotiable.

The implementation roadmap is honest about what can be built in 8 weeks (a functional CLI with local execution and basic routing) versus 8 months (a polished agent runtime with browser

automation, ETD composition, and community exchange). The four-phase structure enforces discipline: each phase has a clear goal, explicit validation criteria, and a gate that must be passed before proceeding. The 33 Codex prompts are designed to be self-contained, sequentially ordered, and independently verifiable, enabling a single human operator to orchestrate the entire development process through iterative agent interactions. The prompts cover all seven agent roles (Spec Architect, Foundation Engineer, Module Implementer, Test Generator, Integration Validator, Security Auditor, ETD Distiller) and span all five subsystems plus the cross-cutting Trust Layer.

The implications of this work extend beyond the NEXUS-R project itself. The agent-native development methodology—where specifications are the primary artifact, agents work in isolation, and verification precedes integration—represents a viable approach for building complex systems entirely through coding agents. The event-sourced development process (where every agent decision is logged as an immutable event) mirrors the event-sourced runtime architecture, creating a pleasing symmetry between the development methodology and the product being developed. If successful, NEXUS-R will demonstrate that agent-native development is not merely a convenience but a discipline that produces more reliable, more auditable, and more maintainable software than traditional human-centric development.

The mandate remains as stated in the v8 specification: Build Phase 1 in 8 weeks. Measure cost reduction. If ETD shows >40% savings on repeated tasks, double down. If not, kill it. A focused 6-module CLI that demonstrably saves users money is worth more than a 12-layer diagram that never compiles. "The AI agent that gets cheaper the more you use it." That is the pitch. That is the moat. Everything else is scaffolding.

13. References

- [1] Xingyao Wang et al., "The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents," arXiv:2511.03690, 2025. Available: <https://arxiv.org/abs/2511.03690>
- [2] Yicheng Guo et al., "Towards Generalized Routing: Model and Agent Orchestration for Adaptive and Efficient Inference (MoMA)," arXiv:2509.07571, 2025. Available: <https://arxiv.org/abs/2509.07571>
- [3] FlowMind Team, "FlowMind: Execute-Summarize for Structured Workflow Generation," arXiv:2602.11782, 2026. Available: <https://arxiv.org/abs/2602.11782>
- [4] Shepherd Team, "Shepherd: A Runtime Substrate Empowering Meta-Agents with a Functional Programming Model," arXiv:2605.10913, 2026. Available: <https://arxiv.org/html/2605.10913v1>
- [5] Anthropic, "Introducing the Model Context Protocol," 2024. Available: <https://anthropic.com/news/model-context-protocol>
- [6] LiteLLM, "Unified Gateway for Managing Enterprise LLM Access," 2025. Available: <https://docs.litellm.ai>
- [7] OpenAI, "Introducing Codex," 2025. Available: <https://openai.com/index/introducing-codex>
- [8] NEXUS-R Project, "NEXUS-R Master Specification v8.0: Unified Synthesis of Implementation Plan + v6/v7 Assessment & Redesign," Internal Document, 2026-05-18.
- [9] NEXUS-R Project, "Agent-Driven Execution Plan: Agent-Native Development," Internal Document, 2026-05-18.